

Chapter 5: Input/Output

Questions and Answers

Question 1

Advances in chip technology have made it possible to put an entire controller, including all the bus access logic, on an inexpensive chip. How does that affect the model of Fig. 1-6?

Answer 1

It becomes practical to build a controller directly into each device. The older model separated controllers so one controller could manage several devices. If controllers are cheap, integrated per-device controllers simplify design and may allow more parallel transfers.

Question 2

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner at 300 dpi	1 MB/sec
Digital camcorder	3.5 MB/sec
4x Blu-ray disc	18 MB/sec
802.11n Wireless	37.5 MB/sec
USB 2.0	60 MB/sec
FireWire 800	100 MB/sec
Gigabit Ethernet	125 MB/sec
SATA 3 disk drive	600 MB/sec
USB 3.0	625 MB/sec
SCSI Ultra 5 bus	640 MB/sec
Single-lane PCIe 3.0 bus	985 MB/sec
Thunderbolt 2 bus	2.5 GB/sec
SONET OC-768 network	5 GB/sec

Figure 5-1. Some typical device, network, and bus data rates.

Given the speeds listed in Fig. 5-1, is it possible to scan documents from a scanner and transmit them over an 802.11g network at full speed? Defend your answer.

Answer 2

Yes. The scanner's maximum rate is about 400 KB/sec, while 802.11g can provide about 6.75 MB/sec. The network is much faster than the scanner, so it is not the bottleneck.

Question 3

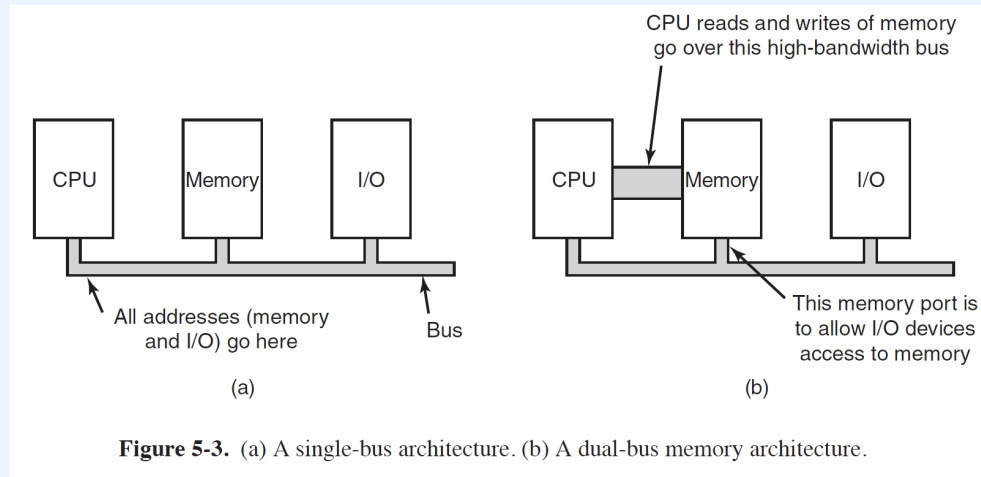


Figure 5-3(b) shows one way of having memory-mapped I/O even in the presence of separate buses for memory and I/O devices, namely, to first try the memory bus and if that fails try the I/O bus. A clever computer science student has thought of an improvement on this idea: try both in parallel, to speed up the process of accessing I/O devices. What do you think of this idea?

Answer 3

No. Normal memory requests would force the CPU to wait for the slower I/O bus attempt to finish or somehow cancel it. This would slow ordinary memory references to I/O-bus speed. The parallel attempt adds complexity and can reduce performance.

Question 4

Explain the tradeoffs between precise and imprecise interrupts on a superscalar machine.

Answer 4

Precise interrupts make the OS simpler because the machine state is clean: all earlier instructions have completed and no later instructions have changed state. Imprecise interrupts simplify high-performance CPU design but force the OS to determine which instructions partially executed.

Tradeoff

Precise interrupts simplify software; imprecise interrupts may simplify or speed hardware.

Question 5

A DMA controller has five channels. The controller is capable of requesting a 32-bit word every 40 nsec. A response takes equally long. How fast does the bus have to be to avoid being a bottleneck?

Answer 5

One transfer needs:

$$40 + 40 = 80 \text{ nsec}$$

Transfers per second:

$$\frac{1}{80 \text{ nsec}} = 12.5 \times 10^6$$

Each transfer is 4 bytes:

$$12.5 \times 10^6 \times 4 = 50 \times 10^6 \text{ bytes/sec}$$

$$\boxed{50 \text{ MB/sec}}$$

Question 6

Suppose that a system uses DMA for data transfer from disk controller to main memory. Further assume that it takes t_1 nsec on average to acquire the bus and t_2 nsec to transfer one word over the bus ($t_1 \gg t_2$). After the CPU has programmed the DMA controller, how long will it take to transfer 1000 words from the disk controller to main memory, if (a) word-at-a-time mode is used, (b) burst mode is used? Assume that commanding the disk controller requires acquiring the bus to send one word and acknowledging a transfer also requires acquiring the bus to send one word.

Answer 6

In **word-at-a-time mode**, each word needs three bus activities:

$$1000 \times [(t_1 + t_2) + (t_1 + t_2) + (t_1 + t_2)]$$

The first term is for acquiring the bus and sending the command to the disk controller, the second term is for transferring the word, and the third term is for the acknowledgement. Therefore:

$$\boxed{3000(t_1 + t_2) \text{ nsec}}$$

In **burst mode**, the command and acknowledgement are separate, but the 1000 data words are transferred in one burst:

$$(t_1 + t_2) + t_1 + 1000t_2 + (t_1 + t_2)$$

The first term is for acquiring the bus and sending the command to the disk controller, the second term is for the disk controller to acquire the bus, the third term is for the burst transfer, and the fourth term is for acquiring the bus and doing the acknowledgement. Therefore:

$$\boxed{3t_1 + 1002t_2 \text{ nsec}}$$

Burst mode is much faster when bus acquisition dominates.

Question 7

One mode that some DMA controllers use is to have the device controller send the word to the DMA controller, which then issues a second bus request to write to memory. How can this mode be used to perform memory to memory copy? Discuss any advantage or disadvantage of

using this method instead of using the CPU to perform memory to memory copy.

Answer 7

The DMA controller can first read a word from one memory address into the controller, then write it to another memory address. The advantage is that the CPU can do other work while the copy proceeds. The disadvantage is that DMA memory-to-memory copying may be slower than CPU copying and consumes system bus bandwidth.

Question 8

Suppose that a computer can read or write a memory word in 5 nsec. Also suppose that when an interrupt occurs, all 32 CPU registers, plus the program counter and PSW are pushed onto the stack. What is the maximum number of interrupts per second this machine can process?

Answer 8

An interrupt saves 34 words and return restores 34 words, for 68 word accesses:

$$68 \times 5 = 340 \text{ nsec}$$

$$\frac{1}{340 \text{ nsec}} \approx 2.94 \times 10^6$$

about 2.94 million interrupts/sec

Question 9

CPU architects know that operating system writers hate imprecise interrupts. One way to please the OS folks is for the CPU to stop issuing new instructions when an interrupt is signaled, but allow all the instructions currently being executed to finish, then force the interrupt. Does this approach have any disadvantages? Explain your answer.

Answer 9

Interrupt latency can increase significantly. Modern CPUs may have long-latency operations in flight, so waiting for all active instructions to finish delays interrupt handling and requires extra bookkeeping.

Question 10

```
copy_from_user(buffer, p, count);
if (count == 0) {
    enable_interrupts();
    unblock_user();
} else {
    *printer_data_register = p[0];
    scheduler();
    count = count - 1;
    i = 1;
}
```

(a)

```
while (*printer_status_reg != READY)
    ;
*printer_data_register = p[i];
count = count - 1;
i = i + 1;
acknowledge_interrupt();
return_from_interrupt();
```

(b)

Figure 5-9. Writing a string to the printer using interrupt-driven I/O. (a) Code executed at the time the print system call is made. (b) Interrupt service procedure for the printer.

In Fig. 5-9(b), the interrupt is not acknowledged until after the next character has been output to the printer. Could it have equally well been acknowledged right at the start of the interrupt service procedure? If so, give one reason for doing it at the end, as in the text. If not, why not?

Answer 10

It could be acknowledged at the start, but acknowledging at the end lets the handler first output the next character. If another interrupt occurs immediately afterward, the printer has already started working on the next character, slightly improving throughput. The cost is slightly longer time before interrupts are reenabled.

Question 11

A computer has a three-stage pipeline as shown in Fig. 1-7(a). On each clock cycle, one new instruction is fetched from memory at the address pointed to by the PC and put into the pipeline and the PC advanced. Each instruction occupies exactly one memory word. The instructions already in the pipeline are each advanced one stage. When an interrupt occurs, the current PC is pushed onto the stack, and the PC is set to the address of the interrupt handler. Then the pipeline is shifted right one stage and the first instruction of the interrupt handler is fetched into the pipeline. Does this machine have precise interrupts? Defend your answer.

Answer 11

Yes. The saved PC points to the first instruction not yet fetched/executed, while all earlier instructions have completed. This gives a well-defined restart point, so the interrupt is precise.

Question 12

A typical printed page of text contains 50 lines of 80 characters each. Imagine that a certain printer can print 6 pages per minute and that the time to write a character to the printer's output register is so short it can be ignored. Does it make sense to run this printer using interrupt-driven I/O if each character printed requires an interrupt that takes 50 μsec all-in to service?

Answer 12

Characters per second:

$$\frac{6 \times 50 \times 80}{60} = 400$$

CPU interrupt cost per second:

$$400 \times 50 \mu\text{sec} = 20 \text{ msec}$$

Only 2% of CPU time is spent handling printer interrupts, so interrupt-driven I/O is reasonable.

Question 13

Explain how an OS can facilitate installation of a new device without any need for recompiling the OS.

Answer 13

The OS can use a device-driver table indexed by device number. Each entry contains function pointers for operations such as open, close, read, and write. Installing a device means adding/loading a driver and filling the table entry.

Question 14

In which of the four I/O software layers is each of the following done. (a) Computing the track, sector, and head for a disk read. (b) Writing commands to the device registers. (c) Checking to see if the user is permitted to use the device. (d) Converting binary integers to ASCII for printing.

Answer 14

Task	Layer
Compute track, sector, head	Device driver
Write commands to device registers	Device driver
Check if user may use device	Device-independent OS software
Convert binary integers to ASCII	User-level software

Question 15

A local area network is used as follows. The user issues a system call to write data packets to the network. The operating system then copies the data to a kernel buffer. Then it copies the data to the network controller board. When all the bytes are safely inside the controller, they are sent over the network at a rate of 10 megabits/sec. The receiving network controller stores each bit a microsecond after it is sent. When the last bit arrives, the destination CPU is interrupted, and the kernel copies the newly arrived packet to a kernel buffer to inspect it. Once it has figured out which user the packet is for, the kernel copies the data to the user space. If we assume that each interrupt and its associated processing takes 1 msec, that packets are 1024 bytes (ignore the headers), and that copying a byte takes 1 μ sec, what is the maximum rate at which one process can pump data to another? Assume that the sender is blocked until the work is finished at the receiving side and an acknowledgement comes back. For simplicity, assume that the time to get the acknowledgement back is so small it can be ignored.

Answer 15

Copy time:

$$4 \times 1024 \mu\text{sec} = 4.096 \text{ msec}$$

Interrupt time:

$$2 \text{ msec}$$

Transmit time:

$$\frac{1024 \times 8}{10^7} \approx 0.819 \text{ msec}$$

Total:

$$4.096 + 2 + 0.819 \approx 6.915 \text{ msec}$$

Rate:

$$\frac{1024}{0.006915} \approx 148,000 \text{ bytes/sec}$$

about 148 KB/sec

Question 16

Why are output files for the printer normally spooled on disk before being printed?

Answer 16

Spooling prevents a process from monopolizing the printer while producing output slowly. The process writes to disk quickly, and the printer daemon prints complete jobs later.